

Anexo C

Santiago Rosas Mogollón

Marzo 2025

1. Fundamentos de Correlación Digital de Imágenes (DIC)

La Correlación Digital de Imágenes (DIC, por sus siglas en inglés, *Digital Image Correlation*) es una técnica óptica metrológica no invasiva que permite medir campos de desplazamiento y deformación en la superficie de un material bajo carga. El principio básico de la DIC radica en el seguimiento de patrones espaciales de intensidad de escala de grises distribuidos sobre la muestra a lo largo de una serie de fotogramas secuenciales.

El proceso general consta de los siguientes pilares matemáticos y algorítmicos:

- **Extracción de Características (Feature Detection):** Identificación de puntos de interés únicos en la imagen invariantes a transformaciones afines, ruido y cambios de iluminación.
- **Descripción de Características (Feature Description):** Generación de un vector numérico (descriptor) que codifica la información visual del entorno del punto de interés.
- **Emparejamiento (Matching):** Minimización de una métrica de distancia espacial o binaria entre los vectores descriptores de una imagen de referencia I_0 (estado sin deformar) y una imagen objetivo I_t (estado deformado).

2. Análisis Comparativo de Algoritmos: SIFT vs. ORB

Para el desarrollo experimental de este sistema de medición de deformaciones, se implementaron y evaluaron dos de los algoritmos más representativos en el estado del arte de la visión por computadora: **SIFT** (*Scale-Invariant Feature Transform*) y **ORB** (*Oriented FAST and Rotated BRIEF*).

2.1. SIFT (Scale-Invariant Feature Transform)

SIFT extrae puntos clave detectando extremos en un espacio de escalas construido mediante funciones Gaussianas (DoG, *Difference of Gaussians*). Su descriptor consiste en un vector de 128 dimensiones basado en gradientes locales de orientación de la imagen. **Ventajas:** Extrema robustez ante cambios drásticos de escala, rotaciones complejas y variaciones afines de iluminación. **Desventajas:** Alto costo computacional debido al cálculo de operaciones en coma flotante sobre un vector denso, lo que limita su viabilidad en sistemas de procesamiento en tiempo real de recursos limitados.

2.2. ORB (Oriented FAST and Rotated BRIEF)

ORB surge como una alternativa eficiente y libre de patentes. Combina el detector de esquinas FAST (*Features from Accelerated Segment Test*) modificado para añadir orientación, junto con el descriptor binario BRIEF (*Binary Robust Independent Elementary Features*) rotado analíticamente. **Ventajas:** En lugar de operar con gradientes vectoriales flotantes, ORB construye una

cadena de bits (un string binario) mediante comparaciones directas de intensidad de píxeles. Esto permite evaluar las distancias de emparejamiento utilizando la distancia de *Hamming* a nivel de compuertas lógicas, acelerando exponencialmente el proceso. Es ideal para el monitoreo concurrente en sistemas *real-time*.

3. Postprocesado (SIFT)

El primer script desarrollado tiene como finalidad realizar un análisis estático de alta precisión comparando el estado inicial (pre-deformación) y final de la probeta mediante el uso secuencial de SIFT.

3.1. Flujo Lógico y Funciones Críticas

1. **Lectura en Escala de Grises:** Se cargan los archivos `a2.png` y `a5.png` en arreglos matriciales bidimensionales omitiendo los canales RGB (`cv2.IMREAD_GRAYSCALE`) para optimizar el cómputo de gradientes.
2. **Detección y Cómputo:** Mediante `sift.detectAndCompute()` se obtienen los puntos clave (*keypoints*) y sus respectivos descriptores.
3. **Emparejamiento por Fuerza Bruta (BFMatcher):** Se inicializa un emparejador que evalúa de forma exhaustiva cada descriptor de la primera imagen contra todos los de la segunda mediante una distancia euclidiana L2.
4. **Filtro de Ratio de Lowe:** Para discriminar emparejamientos ambiguos o falsos positivos causados por texturas repetitivas, se aplica el criterio de Lowe:

$$d_{\text{match}_1} < 0,75 \times d_{\text{match}_2} \quad (1)$$

Donde solo se aceptan los puntos cuyo vecino más cercano sea significativamente más próximo que el segundo vecino más cercano.

5. **Cálculo Euclidiano del Desplazamiento:** Una vez aislados los *matches* válidos, se extraen las coordenadas espaciales (x, y) en el plano cartesiano del sensor óptico de ambos estados y se calcula la distancia geométrica:

$$\text{Displacement} = \sqrt{(x_{\text{after}} - x_{\text{before}})^2 + (y_{\text{after}} - y_{\text{before}})^2} \quad (2)$$

Finalmente, se aplica el operador media aritmética (`np.mean`) para estimar el desplazamiento promedio global en píxeles del material.

4. Tiempo Real (ORB)

El segundo desarrollo responde a la necesidad de adquirir, sincronizar y registrar datos de manera síncrona en tiempo real, operando con dos cámaras independientes de forma simultánea y vinculando mediciones analógicas externas de fuerza. El algoritmo ORB se configuró mediante un ajuste fino de sus coeficientes operativos para equilibrar la densidad de características y la velocidad de cómputo en la Región de Interés (ROI). El parámetro `nfeatures=500` restringe la búsqueda a los 500 puntos clave más estables, limitando la carga computacional por ciclo. Para otorgar invariabilidad de escala frente a posibles desplazamientos axiales de las cámaras, se definió un `scaleFactor=1.2` estructurado en `nlevels=8`, lo que genera una pirámide de resolución de 8 capas donde cada nivel se reduce en un 20 % respecto al anterior. El análisis espacial y descarte de ruido en la ROI se controla mediante `edgeThreshold=31` y `patchSize=31`, dimensiones que delimitan el vecindario de píxeles para el cálculo del descriptor binario. La

evaluación de los puntos se realiza a través de la métrica `scoreType=cv2.ORB_HARRIS_SCORE`, la cual aplica el criterio de la matriz de autodiferencia de Harris para clasificar y ordenar los puntos en función de la agudeza de sus esquinas, en lugar del enfoque FAST puro. Finalmente, los parámetros `WTA_K=2` y `fastThreshold=20` definen, respectivamente, que el descriptor BRIEF realice comparaciones binarias de pares de píxeles orientados y que el umbral de aceptación FAST sea lo suficientemente sensible para capturar texturas sutiles en la superficie del material bajo ensayo

4.1. Arquitectura del Sistema en Tiempo Real

El script inicializa un entorno de adquisición concurrente estructurado bajo los siguientes módulos operativos:

- **Inicialización de Hardware Dual:** Se configuran dos instancias de captura de video a una resolución nativa de alta definición (1280×720 píxeles). La primera cámara captura el plano macro de deformación (“Camera meso”), mientras que la segunda registra la perspectiva a nivel microscópico (“Camera micro”). Los flujos de video crudos se almacenan en tiempo real en discos locales en formato contenedor MP4 utilizando la biblioteca `VideoWriter` con códec `mp4v` y compresión controlada al 80 %.
- **Selección Dinámica de la Región de Interés (ROI):** Al arrancar el bucle principal, se congela el primer fotograma y se despliega una interfaz interactiva de usuario (`cv2.selectROI()`). Esto permite aislar un cuadrante específico del material donde se concentran los esfuerzos mecánicos, acotando el espacio de cómputo del extractor ORB de manera drástica y descartando el ruido del entorno de fondo.
- **Mecanismo de Filtrado y Gestión de Ruido:** Para blindar las lecturas métricas de la vibración inherente del motor y las fluctuaciones del sensor, se parametriza un **umbral de desplazamiento** de 1 píxel (`displacement_threshold`). Si el desplazamiento promedio extraído por los vectores de Hamming no supera este límite, el cambio es clasificado como ruido blanco y se conserva el último estado estacionario válido (`last_valid_displacement`).
- **Sincronización Serial y Exportación de Datos:** A intervalos constantes controlados de 0,3 segundos ($\Delta t = 300$ ms), el algoritmo interrumpe la captura visual para decodificar las tramas de datos del puerto de comunicación serial COM4 conectado al microcontrolador Arduino. Se extraen dos variables críticas: el valor analógico crudo y la fuerza calculada en Newtons. Estos parámetros se empaquetan instantáneamente junto con una marca temporal de precisión milimétrica (*Timestamp*) dentro de una hoja de cálculo relacional estructurada con la biblioteca `openpyxl`.
- **Interrupción de Flujo y Control de Referencias:** El operador cuenta con control de flujo por teclado. Presionar la tecla ‘p’ conmuta el sistema a un estado de suspensión visual (*Pausa*) que detiene la grabación en archivo. Al reanudar la marcha, el script obliga de manera automática a redefinir una nueva ROI y calcular un nuevo vector descriptor de referencia espacial, permitiendo un análisis de deformación por etapas acumulativas sin arrastrar errores de deriva.

Apéndice I: Código Fuente del Script de Postprocesado (SIFT)

```
1 import cv2
2 import numpy as np
3
4 # Cargar imagenes antes y despues de la deformacion
5 image_before = cv2.imread('a2.png', cv2.IMREAD_GRAYSCALE)
6 image_after = cv2.imread('a5.png', cv2.IMREAD_GRAYSCALE)
7
8 # Inicializa SIFT
9 sift = cv2.SIFT_create()
10
11 # Detecta puntos caracteristicos y descriptores en ambas imagenes
12 keypoints_before, descriptors_before = sift.detectAndCompute(
    image_before, None)
13 keypoints_after, descriptors_after = sift.detectAndCompute(image_after,
    None)
14
15 # Empareja descriptores usando el matcher BF (Brute-Force)
16 bf = cv2.BFMatcher()
17 matches = bf.knnMatch(descriptors_before, descriptors_after, k=2)
18
19 # Filtra matches usando el ratio de Lowe
20 good_matches = []
21 for m, n in matches:
22     if m.distance < 0.75 * n.distance:
23         good_matches.append(m)
24
25 # Dibuja los matches
26 output_image = cv2.drawMatches(image_before, keypoints_before,
    image_after, keypoints_after, good_matches, None, flags=2)
27
28 # Redimensionar la imagen de salida para que quepa en la ventana
29 scale_percent = 50 # Porcentaje del tamaño original
30 width = int(output_image.shape[1] * scale_percent / 100)
31 height = int(output_image.shape[0] * scale_percent / 100)
32 dim = (width, height)
33 resized_output_image = cv2.resize(output_image, dim, interpolation=cv2.
    INTER_AREA)
34
35 # Crear una ventana con un tamaño específico
36 cv2.namedWindow('Matches', cv2.WINDOW_NORMAL)
37 cv2.resizeWindow('Matches', width, height)
38
39 # Mostrar los matches
40 cv2.imshow('Matches', resized_output_image)
41 cv2.waitKey(0)
42 cv2.destroyAllWindows()
43
44 # Calcula desplazamientos
45 displacements = []
46 for match in good_matches:
47     pt_before = keypoints_before[match.queryIdx].pt
48     pt_after = keypoints_after[match.trainIdx].pt
49     displacement = np.sqrt((pt_after[0] - pt_before[0])**2 + (pt_after
    [1] - pt_before[1])**2)
50     displacements.append(displacement)
51
```

```
52 # Calcula la deformacion promedio
53 average_displacement = np.mean(displacements)
54 print(f"Desplazamiento promedio: {average_displacement} pixeles")
```

Apéndice II: Código Fuente del Script en Tiempo Real (ORB)

```
1 import cv2
2 import numpy as np
3 import time
4 from openpyxl import Workbook
5 from datetime import datetime
6 import serial
7 # Configuración de visualización
8 DISPLAY_WIDTH = 640
9 DISPLAY_HEIGHT = 480
10 # resolución
11 RESOLUCION = (1280, 720)
12 CALIDAD = 80
13 # Inicializar ORB
14 orb = cv2.ORB_create(nfeatures=500,
15                      scaleFactor=1.2,
16                      nlevels=8,
17                      edgeThreshold=31,
18                      firstLevel=0,
19                      WTA_K=2,
20                      scoreType=cv2.ORB_HARRIS_SCORE,
21                      patchSize=31,
22                      fastThreshold=20)
23
24 # Inicializar la cámara principal
25 cap = cv2.VideoCapture(1) # Usar la cámara predeterminada (cámara web)
26
27 # Inicializar la segunda cámara
28 cap2 = cv2.VideoCapture(0) # Usar la segunda cámara (ajusta el índice
29                             según tu configuración)
30
31 # Verificar si las cámaras se abrieron correctamente
32 if not cap.isOpened():
33     print("Error: No se pudo abrir la cámara principal.")
34     exit()
35
36 if not cap2.isOpened():
37     print("Error: No se pudo abrir la segunda cámara.")
38     exit()
39
40 # Configurador de resolución a 1080p para ambas cámaras
41 cap.set(cv2.CAP_PROP_FRAME_WIDTH, RESOLUCION[0])
42 cap.set(cv2.CAP_PROP_FRAME_HEIGHT, RESOLUCION[1])
43
44 cap2.set(cv2.CAP_PROP_FRAME_WIDTH, RESOLUCION[0])
45 cap2.set(cv2.CAP_PROP_FRAME_HEIGHT, RESOLUCION[1])
46
47 # Obtener la resolución del frame de la cámara principal
48 frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
49 frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
50 fps = int(cap.get(cv2.CAP_PROP_FPS)) # Cuadros por segundo (ajustar
51                                     según la cámara)
52
53 # Obtener la resolución del frame de la segunda cámara
54 frame_width2 = int(cap2.get(cv2.CAP_PROP_FRAME_WIDTH))
55 frame_height2 = int(cap2.get(cv2.CAP_PROP_FRAME_HEIGHT))
56 fps2 = int(cap2.get(cv2.CAP_PROP_FPS))
```

```

54 # Inicializar VideoWriter para guardar el video de la cámara principal
55 video_output = cv2.VideoWriter('salida_video_macro.mp4', cv2.
    VideoWriter_fourcc(*'mp4v'), fps, RESOLUCION)
56 video_output.set(cv2.VIDEOWRITER_PROP_QUALITY, CALIDAD)
57 # Inicializar VideoWriter para guardar el video de la segunda cámara
58 video_output2 = cv2.VideoWriter('salida_video_micro.mp4', cv2.
    VideoWriter_fourcc(*'mp4v'), fps2, RESOLUCION)
59 video_output2.set(cv2.VIDEOWRITER_PROP_QUALITY, CALIDAD)
60
61 # Variables para almacenar el frame de referencia
62 reference_frame = None
63 reference_keypoints = None
64 reference_descriptors = None
65 roi = None # Variable para almacenar la ROI
66
67 # Variable para controlar el tiempo
68 last_time = time.time()
69 interval = 0.3 # Intervalo de 0.3 segundo
70
71 # Umbral para ignorar desplazamientos pequeños (en píxeles)
72 displacement_threshold = 1 # Ignorar desplazamientos menores a 1 píxel
73
74 # Crear un archivo Excel para guardar los datos
75 wb = Workbook()
76 ws = wb.active
77 ws.title = "Deformación"
78 ws.append(["Timestamp", "Desplazamiento (píxeles)", "Valor leído", "
    Newtons", "Tiempo arduino", "Tiempo arduino date", "Tiempo de ciclo",
    "Tiempo final"]) # Encabezados
79
80 # Variable para controlar la pausa
81 paused = False
82
83
84
85 # Función para capturar el frame de referencia y seleccionar la ROI
86 def capture_reference_frame():
87     global reference_frame, reference_keypoints, reference_descriptors,
    roi
88     ret, frame = cap.read()
89     if not ret:
90         print("Error: No se pudo capturar el frame de referencia.")
91         return False
92
93     # Mostrar el frame y permitir al usuario seleccionar la ROI
94     roi = cv2.selectROI("Selecciona la ROI", frame, fromCenter=False,
    showCrosshair=True)
95     cv2.destroyWindow("Selecciona la ROI") # Cerrar la ventana de
    selecci n de ROI
96
97     # Recortar el frame de referencia según la ROI
98     reference_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
99     reference_frame = reference_frame[int(roi[1]):int(roi[1] + roi[3]),
    int(roi[0]):int(roi[0] + roi[2])]
100
101     # Detectar puntos característicos y descriptores en la ROI
102     reference_keypoints, reference_descriptors = orb.detectAndCompute(
    reference_frame, None)

```

```

103     if reference_descriptors is None:
104         print("Error: No se pudieron calcular los descriptores de
referencia.")
105         return False
106     print("Frame de referencia capturado y ROI seleccionada.")
107     return True
108
109 # Capturar el frame de referencia y seleccionar la ROI inicial
110 if not capture_reference_frame():
111     cap.release()
112     cap2.release()
113     cv2.destroyAllWindows()
114     exit()
115
116 # Variable para almacenar el ltimo desplazamiento v lido
117 # Variable para controlar el tiempo
118 last_time = time.time()
119 interval = 0.2 # Intervalo de 0.1 segundo
120 last_valid_displacement = 0.0
121 # Configura el puerto serial (ajusta el puerto y el baud rate seg n tu
configuraci n)
122 newtons = 0.0
123 valor_leido = 0.0
124 puerto_serial = serial.Serial('COM4', 115200, timeout=1)# Cambia 'COMX'
por el puerto correcto
125 print(datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f"))
126 print(time.time())
127 while True:
128     time_dateardu = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")
129     timeardu = time.time()
130     # Capturar un frame de la c mara principal
131     ret, frame = cap.read()
132     if not ret:
133         print("Error: No se pudo capturar el frame de la c mara
principal.")
134         break
135     frame_display = cv2.resize(frame, (DISPLAY_WIDTH, DISPLAY_HEIGHT))
136     # Capturar un frame de la segunda c mara
137     ret2, frame2 = cap2.read()
138     if not ret2:
139         print("Error: No se pudo capturar el frame de la segunda c mara
.")
140         break
141     frame2_display = cv2.resize(frame2, (DISPLAY_WIDTH, DISPLAY_HEIGHT))
142     # Dibujar la ROI en el frame de la c mara principal para
visualizaci n (no se guarda en el video)
143     frame_with_roi = frame.copy()
144     if roi is not None:
145         cv2.rectangle(frame_with_roi, (int(roi[0]), int(roi[1])), (int(
roi[0] + roi[2]), int(roi[1] + roi[3])), (0, 255, 0), 2)
146
147     # Mostrar el estado de pausa en la ventana de la c mara principal
148     if paused:
149         cv2.putText(frame_with_roi, "PAUSADO", (10, 50), cv2.
FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2, cv2.LINE_AA)
150
151     # Mostrar el frame con la ROI en tiempo real
152     cv2.imshow('Camera meso', frame_with_roi)

```



```

153
154     # Mostrar el frame de la segunda c mara en una ventana separada
155     if paused:
156         # Si est en pausa, mostrar un mensaje de "PAUSADO" en la
segunda c mara
157         frame2_paused = frame2.copy()
158         cv2.putText(frame2_paused, "PAUSADO", (10, 50), cv2.
FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2, cv2.LINE_AA)
159         cv2.imshow('Camera micro', frame2_paused)
160     else:
161         # Si no est en pausa, mostrar el frame normal
162         cv2.imshow('Camera micro', frame2)
163
164     # Guardar el frame original (sin la ROI) en el video de la c mara
principal si no est en pausa
165     if not paused:
166         video_output.write(frame)
167         video_output2.write(frame2) # Guardar el frame de la segunda
c mara
168
169     # Obtener el tiempo actual
170     current_time = time.time()
171
172     # Procesar solo si ha pasado 1 segundo desde el ltimo
procesamiento y no est en pausa
173     if not paused and current_time - last_time >= interval:
174         # Recortar el frame seg n la ROI
175         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
176         gray_roi = gray[int(roi[1]):int(roi[1] + roi[3]), int(roi[0]):
int(roi[0] + roi[2])]
177
178         # Detectar puntos caracter sticos y descriptores en la ROI del
frame actual
179         keypoints, descriptors = orb.detectAndCompute(gray_roi, None)
180
181         # Verificar si los descriptores se calcularon correctamente
182         if descriptors is not None and reference_descriptors is not None
:
183             # Emparejar descriptores usando BFMatcher
184             bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
185             matches = bf.match(reference_descriptors, descriptors)
186
187             # Calcular el desplazamiento promedio en p xeles
188             average_displacement = 0.0 # Valor predeterminado si no hay
matches
189             if len(matches) > 0:
190                 # Ordenar matches por distancia
191                 matches = sorted(matches, key=lambda x: x.distance)
192
193                 # Calcular desplazamientos
194                 displacements = []
195                 for match in matches:
196                     pt1 = reference_keypoints[match.queryIdx].pt
197                     pt2 = keypoints[match.trainIdx].pt
198                     dx = pt2[0] - pt1[0]
199                     dy = pt2[1] - pt1[1]
200                     displacement = np.sqrt(dx**2 + dy**2)
201                     displacements.append(displacement)

```

```

202         # Calcular el desplazamiento promedio
203         average_displacement = np.mean(displacements)
204
205         # Solo mostrar el desplazamiento si supera el umbral
206         if average_displacement > displacement_threshold:
207             print(f"Desplazamiento promedio: {
208 average_displacement:.2f} p xeles")
209             last_valid_displacement = average_displacement #
Actualizar el ltimo desplazamiento v lido
210         else:
211             print("Desplazamiento despreciable (ruido)")
212             average_displacement = last_valid_displacement #
Usar el ltimo desplazamiento v lido
213         else:
214             print("No se encontraron matches. Usando el ltimo
desplazamiento v lido.")
215             average_displacement = last_valid_displacement # Usar
el ltimo desplazamiento v lido
216
217         # Dibujar los matches (si los hay)
218         if len(matches) > 0:
219             ref_resized = cv2.resize(reference_frame, (DISPLAY_WIDTH
// 2, DISPLAY_HEIGHT // 2))
220             current_resized = cv2.resize(gray_roi, (DISPLAY_WIDTH //
2, DISPLAY_HEIGHT // 2))
221             output_frame = cv2.drawMatches(ref_resized,
reference_keypoints, current_resized, keypoints, matches[:50], None,
flags=2)
222             output_frame = cv2.resize(output_frame, (DISPLAY_WIDTH,
DISPLAY_HEIGHT // 2))
223             cv2.imshow('Matches', output_frame)
224         else:
225             print("No se pudieron calcular los descriptores. Usando el
ltimo desplazamiento v lido.")
226             average_displacement = last_valid_displacement # Usar el
ltimo desplazamiento v lido
227
228         # Leer datos del Arduino
229         linea = puerto_serial.readline().decode('utf-8').strip()
230         if linea:
231             partes = linea.split()
232             if len(partes) >= 5: # Asegurarse de que hay suficientes
elementos
233                 valor_leido = float(partes[2]) # El valor le do es el
tercer elemento
234                 newtons = float(partes[4]) # Los Newtons son el sexto
elemento
235                 print(f"Fuerza:{newtons:.2f}")
236                 # Guardar el desplazamiento y los datos del Arduino en
el archivo Excel
237                 timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%
f")
238                 finali_time = time.time()
239                 ws.append([timestamp, average_displacement, valor_leido,
newtons, time_ardu, time_date_ardu, current_time, finali_time])
240
241         # Reiniciar el temporizador

```

```

242     last_time = current_time
243     # Manejar la tecla 'p' para pausar/continuar
244     key = cv2.waitKey(1)
245     if key & 0xFF == ord('p'):
246         paused = not paused
247         if not paused:
248             # Al continuar, capturar un nuevo frame de referencia y
249             seleccionar una nueva ROI
250             if not capture_reference_frame():
251                 break # Salir si no se pudo capturar el frame de
252             referencia
253             print("Pausado" if paused else "Continuando con nueva referencia
254             ")
255
256     # Salir con la tecla 'q'
257     if key & 0xFF == ord('q'):
258         break
259
260 # Guardar el archivo Excel
261 output_file = "análisis_deformación2.xlsx"
262 wb.save(output_file)
263 print(f"Datos guardados en {output_file}")
264
265 # Liberar la cámara principal, el VideoWriter y cerrar ventanas
266 cap.release()
267 video_output.release()
268
269 # Liberar la segunda cámara y su VideoWriter
270 cap2.release()
271 video_output2.release()
272
273 cv2.destroyAllWindows()
274
275 # Cierra el puerto serial
276 puerto_serial.close()

```